Serverless Image Processing Pipeline on AWS

Objective

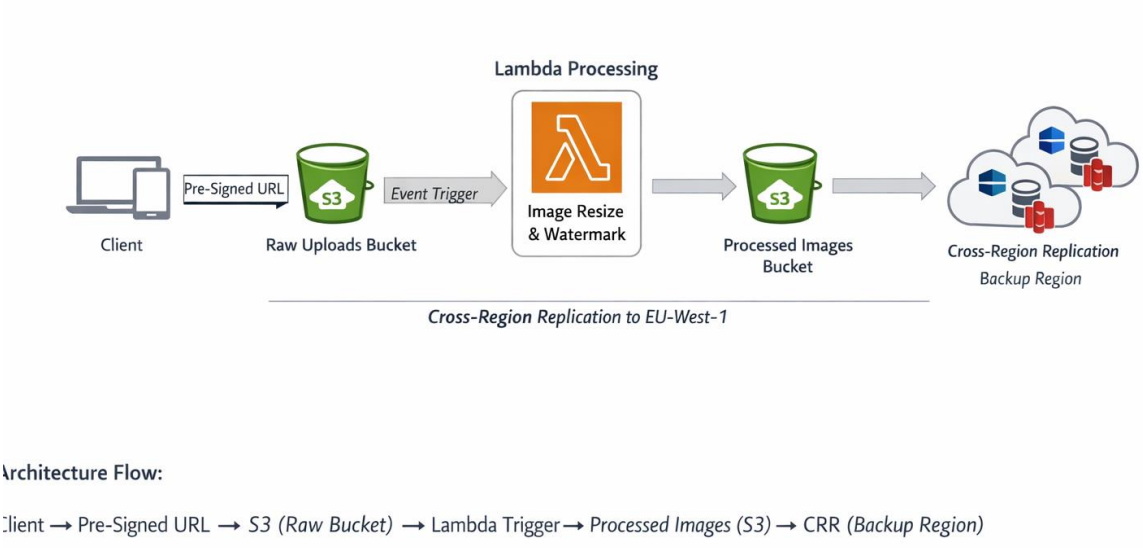
The objective of this project is to build a **scalable and automated image processing pipeline** using AWS services.

This includes:

- Secure image upload using Pre-Signed URLs
- Automated image processing using Lambda
- Storage management using S3
- Data protection using encryption and cross-region replication

Architecture

Client → Pre-Signed URL → S3 (Raw Bucket) → Lambda Trigger → Processed Images (S3) → CRR



Architecture Flow:

Client → Pre-Signed URL → *S3 (Raw Bucket)* → Lambda Trigger → *Processed Images (S3)* → CRR *(Backup Region)*

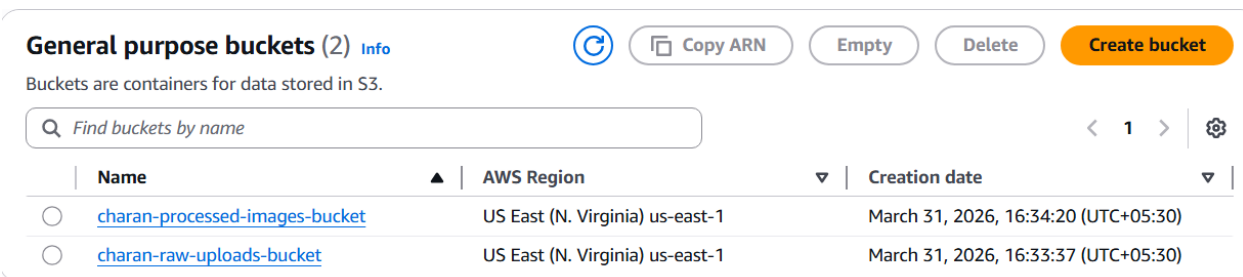
1. S3 Setup

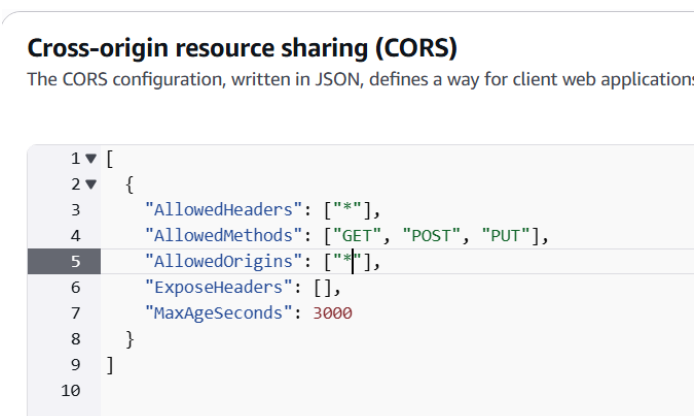
Configuration Details:

- Create **raw-uploads-bucket**
  - Enable Versioning
  - Enable SSE-S3 Encryption
  - Configure CORS
- Create **processed-images-bucket**
  - Enable SSE-KMS Encryption
- Enable **Cross-Region Replication (CRR)** to **eu-west-1**
- Configure **Lifecycle Rules**
  - Transition objects to cheaper storage tiers

Explanation:

Provides secure storage, version control, disaster recovery, and cost optimization.





size filter for each applicable lifecycle rule with a transition action.

Transition current versions of objects between storage classes

Choose transitions to move current versions of objects between storage classes based on your use case scenario and performance access requirements. These transitions start from when the objects are cr
consecutively applied. Learn more

Choose storage class transitions	Days after object creation	
Standard-IA	30	Remove
Glacier Flexible Retrieval (formerly Glacier)	60	Remove
Glacier Deep Archive	180	Remove

Add transition

Expire current versions of objects

For version-enabled buckets, Amazon S3 adds a delete marker and the current version of an object is retained as a noncurrent version. For non-versioned buckets, Amazon S3 permanently removes the ob

Days after object creation

365

Amazon S3 > Buckets > charan-raw-uploads-bucket > Lifecycle configuration

Lifecycle configuration

To manage your objects so that they are stored cost effectively throughout their lifecycle, configure their lifecycle. A lifecycle configuration is a set of rules that define actions that Amazon S3 applies to a group of objects. Lifecycle rules run once per day.

Default minimum object size for transitions
All storage classes 128K

Lifecycle rules (1)

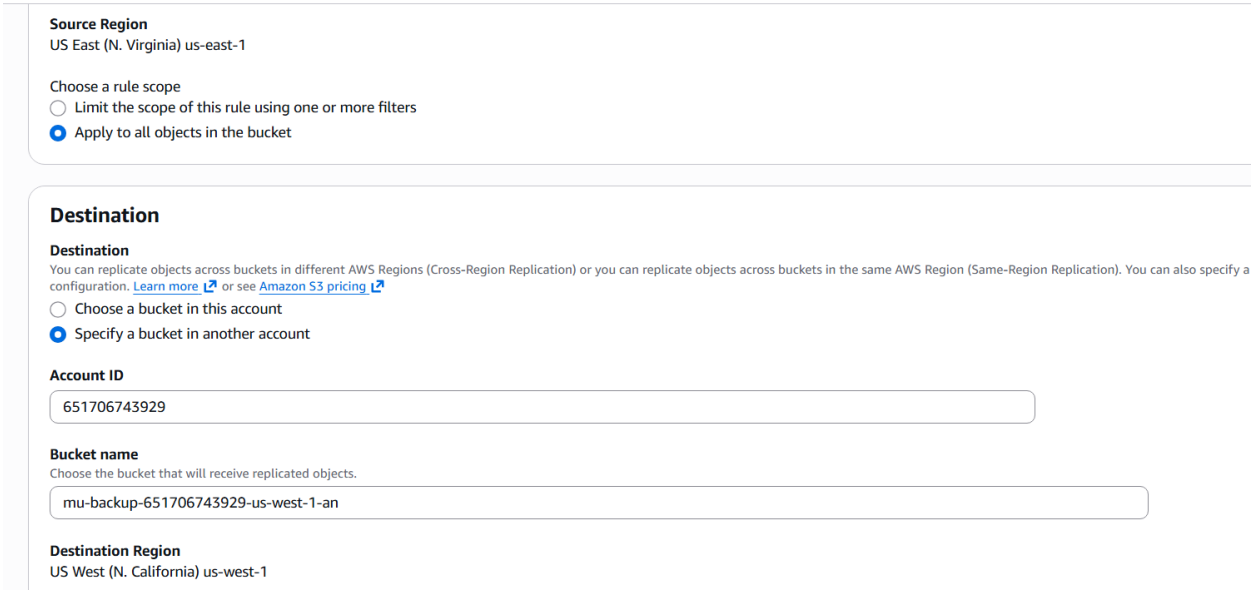
View details | Edit | Delete | Actions | Create lifecycle rule

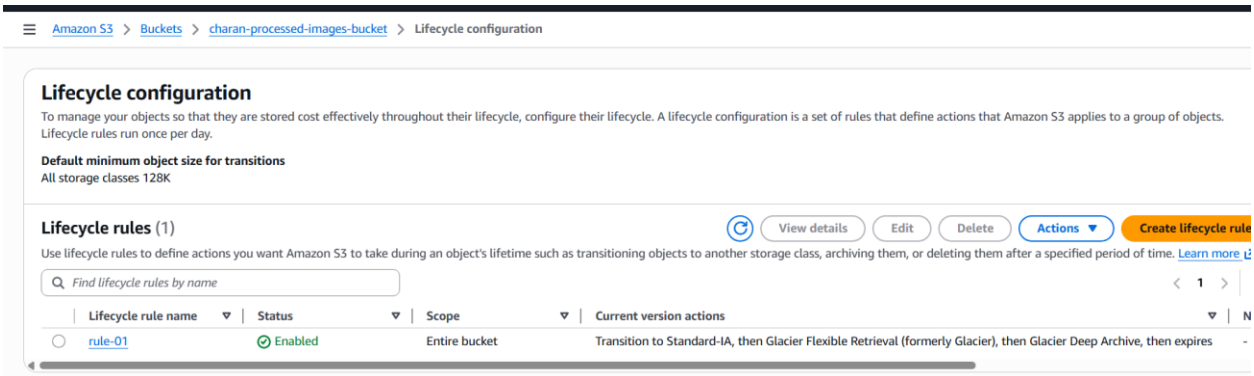
Use lifecycle rules to define actions you want Amazon S3 to take during an object's lifetime such as transitioning objects to another storage class, archiving them, or deleting them after a specified period of time. Learn more

Find lifecycle rules by name

1

Lifecycle rule name	Status	Scope	Current version actions
rule-01	Enabled	Entire bucket	Transition to Standard-IA, then Glacier Flexible Retrieval (formerly Glacier), then Glacier Deep Archive, then expires





2. Lambda Processing Function

Configuration Details:

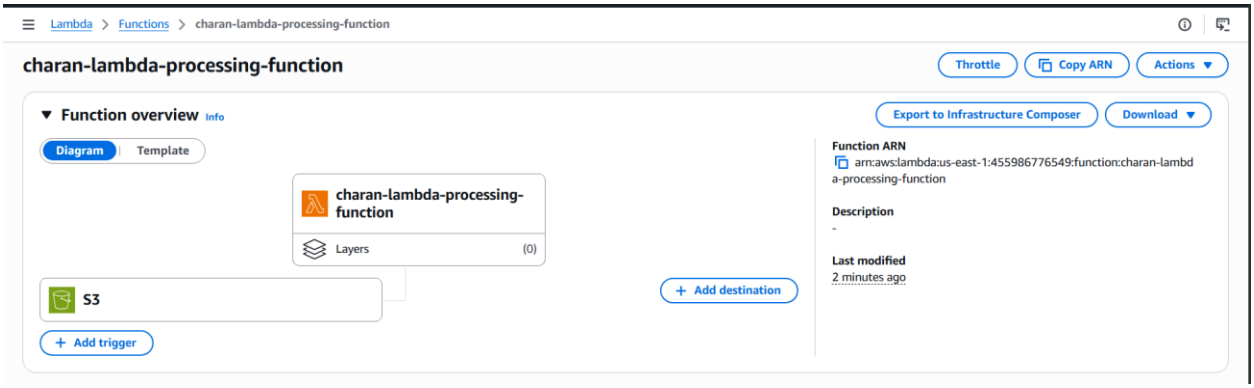
- Runtime: **Python 3.11**
- Memory: **512 MB**
- Timeout: **30 seconds**
- Trigger: S3 (PUT Object)

Steps:

1. Create Lambda function
2. Add S3 trigger (raw-uploads-bucket)
3. Configure memory and timeout

Explanation:

Automatically processes images when uploaded.



3. Lambda Layer (Pillow Setup)

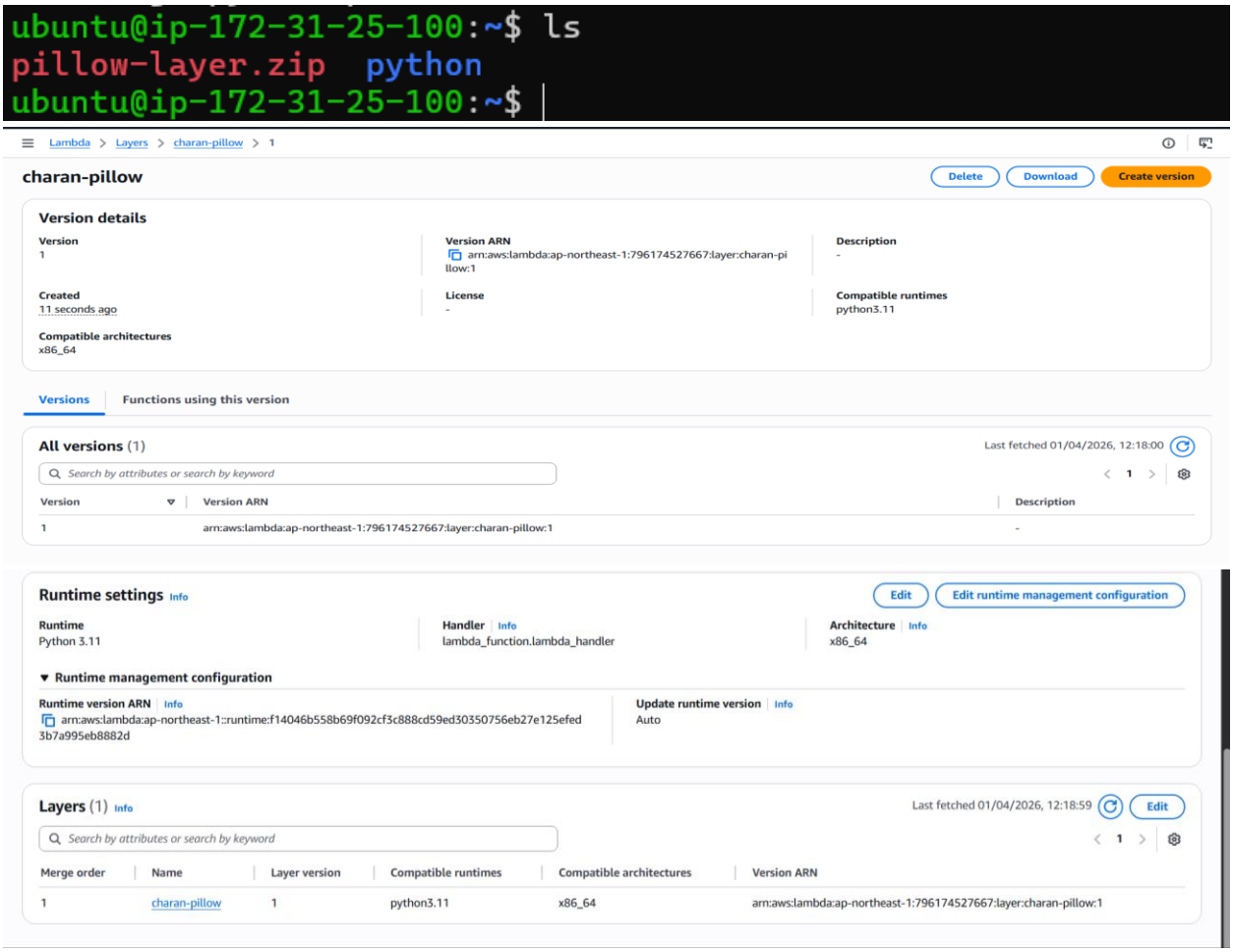
Steps:

```
mkdir python
pip install pillow -t python/
zip -r pillow-layer.zip python
```

- Upload as Lambda Layer
- Attach to function

Explanation:

Provides image processing capability.



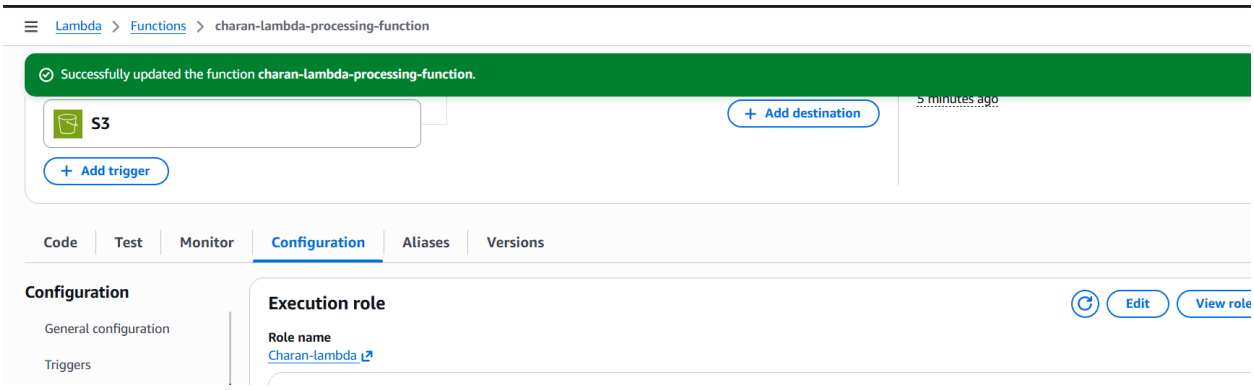
4. IAM Permissions

Required Permissions:

- `s3:GetObject`
- `s3:PutObject`
- [AWSLambdaBasicExecutionRole]

Explanation:

Allows Lambda to read/write images between buckets.



5. Image Processing Logic

Function Logic:

- Resize to **800x600**
- Add watermark
- Save to processed bucket

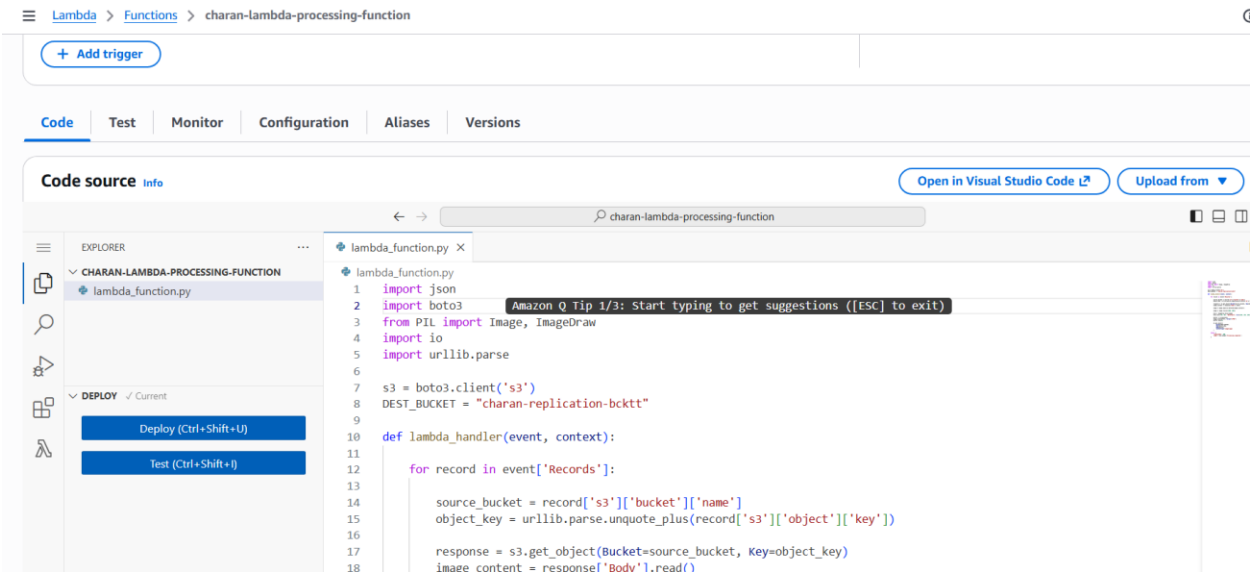
Code:

```
import json
import boto3
from PIL import Image, ImageDraw
import io
import urllib.parse

s3 = boto3.client('s3')
DEST_BUCKET = "processed-images-bucket"

def lambda_handler(event, context):

    for record in event['Records']:

        source_bucket = record['s3']['bucket']['name']
        object_key =
urllib.parse.unquote_plus(record['s3']['object']['key'])

        response = s3.get_object(Bucket=source_bucket, Key=object_key)
        image_content = response['Body'].read()
```

```
        image = Image.open(io.BytesIO(image_content))

        image = image.resize((800, 600))

        draw = ImageDraw.Draw(image)
        draw.text((10, 10), "Watermark", fill=(255, 255, 255))

        buffer = io.BytesIO()
        image.save(buffer, format="JPEG")
        buffer.seek(0)

        s3.put_object(
            Bucket=DEST_BUCKET,
            Key=object_key,
            Body=buffer,
            ContentType='image/jpeg'
        )

    return {
        'statusCode': 200,
        'body': json.dumps('Processing complete')
    }
```

Explanation:

Handles resizing, watermarking, and upload automatically.



6. Pre-Signed URL Generator

Configuration Details:

- Lambda function generates **PUT URL**
- Validity: **15 minutes (900 seconds)**

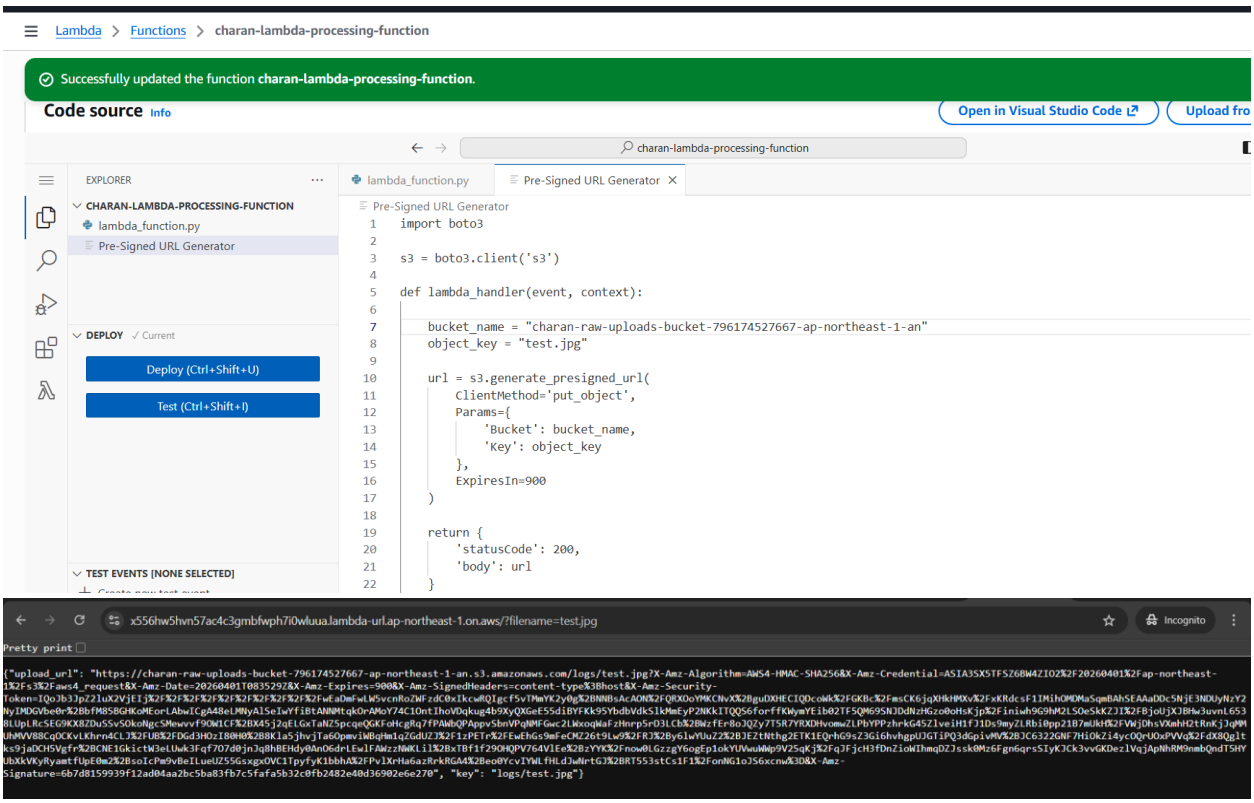
Code:

```
import boto3

s3 = boto3.client('s3')

def lambda_handler(event, context):

    bucket_name = "raw-uploads-bucket"
    object_key = "test.jpg"

    url = s3.generate_presigned_url(
        ClientMethod='put_object',
        Params={
            'Bucket': bucket_name,
            'Key': object_key
        },
```

```
        ExpiresIn=900
    )

    return {
        'statusCode': 200,
        'body': url
    }
```

Explanation:

Generates a secure temporary URL for uploading images without exposing AWS credentials.



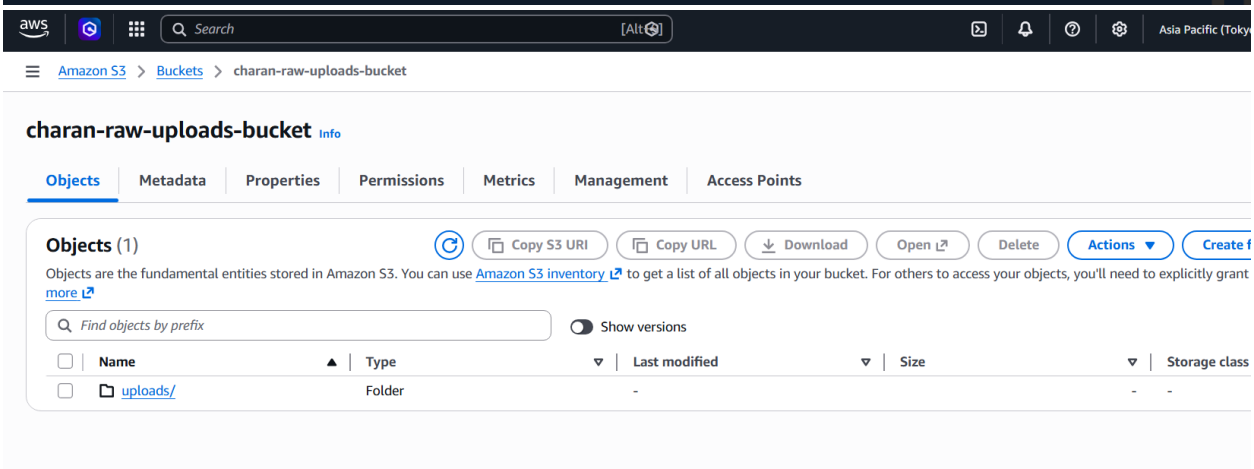
7. Testing

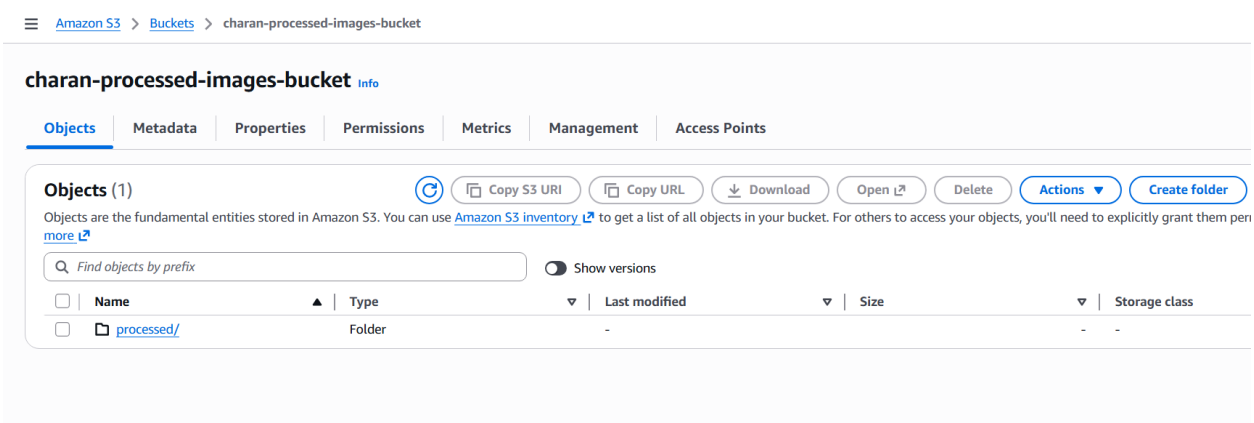
Upload using curl:

```
curl -X PUT -T test.jpg "PASTE_URL_HERE"
```

Expected Result:

- Image uploaded to raw bucket
- Lambda triggered
- Processed image stored in processed bucket





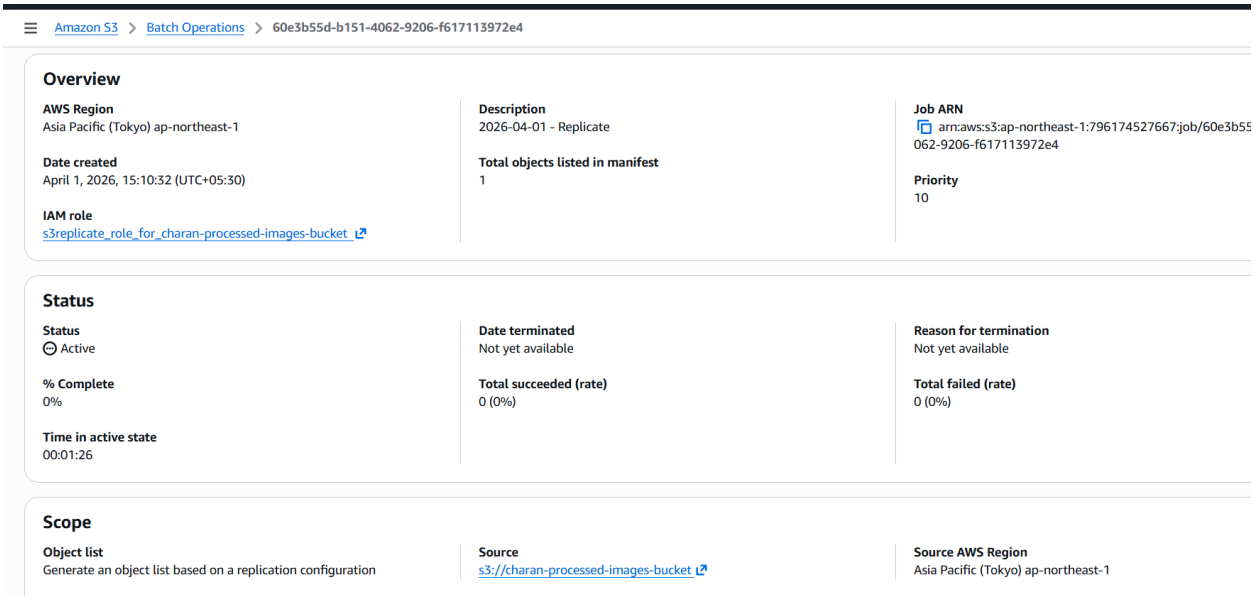
8. Storage Validation

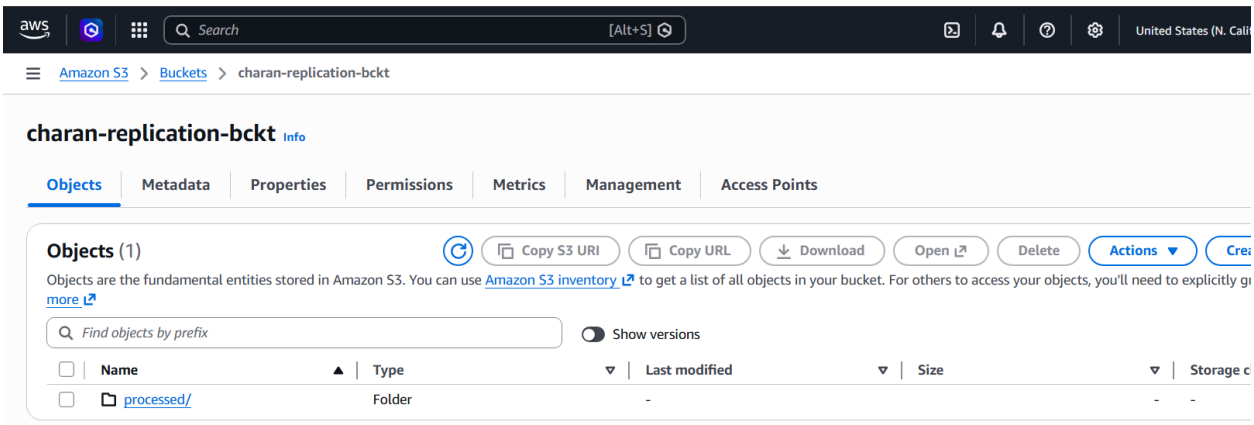
Validation Tasks:

- Verify lifecycle transitions (CLI / S3 console)
- Check CRR replication in **eu-west-1**

Explanation:

Ensures storage optimization and replication are working.





Key Learnings

✓ Pre-Signed URLs

✓ S3 CORS & Lifecycle Rules

✓ Lambda triggers

✓ Image processing using Pillow

✓ Cross-Region Replication (CRR)

✓ Encryption (SSE-S3 & SSE-KMS)

Final Outcome

This solution provides:

- Secure file upload mechanism
- Automated serverless image processing
- Cost-efficient storage lifecycle
- Disaster recovery using CRR